Translated from English to Vietnamese - www.onlinedoctranslator.com

6.087 Bài giảng số 3 – Ngày 13 tháng 1 năm 2010

- Ôn tập
- Khối và câu phức
- Luồng điều khiển
  - Câu lệnh điều kiện
  - Vòng lặp
- Chức năng
- Lập trình mô-đun
- Phạm vi biến đổi
  - Biến tĩnh
  - Đăng ký biến

Ôn tập: Định nghĩa

- Biến - tên/tham chiếu đến một giá trị được lưu trữ (thường là trong bộ nhớ)
- Kiểu dữ liệu - xác định kích thước của biến trong bộ nhớ, các giá trị mà nó có thể nhận, và các phép toán được cho phép.
- Toán tử - một phép toán được thực hiện bằng cách sử dụng 1-3 biến
- Biểu thức - sự kết hợp giữa các giá trị/biến cố định và các toán tử/hàm.

Ôn tập: Các loại dữ liệu

- Nhiều kích cỡ khác nhau (**ký tự**,**ngắn**,**dài**,**flyến mạch**,**gấp đôi**)
- Các kiểu dữ liệu số -**đã ký**/**chưa ký**
- Cách triển khai - little endian hoặc big endian
- Trộn và đúc các loại vật liệu một cách cẩn thận.

Đánh giá: Nhà mạng

- Một ngôi, hai ngôi, ba ngôi (1-3 đối số)
- Các toán tử số học, toán tử quan hệ, toán tử nhị phân (toán tử bit và toán tử logic), toán tử gán, v.v.
- Biểu thức điều kiện
- Thứ tự đánh giá (ưu tiên, hướng)

- Ôn tập
- Khối và câu phức
- Luồng điều khiển
  - Câu lệnh điều kiện
  - Vòng lặp
- Chức năng
- Lập trình mô-đun
- Phạm vi biến đổi
  - Biến tĩnh
  - Đăng ký biến

Khối và câu phức

- Một câu đơn giản kết thúc bằng dấu chấm phẩy:

  z = foo(x+y);

- Hãy xem xét nhiều phát biểu sau:

  nhiệt độ = x + y;
  z = foo (temp) ;

- Dấu ngo ngoặc nhọn – kết hợp thành câu phức/*khối*

Khối

- Khối lệnh có thể thay thế cho câu phát biểu đơn giản.
- Được biên soạn thành một đơn vị duy nhất
- Biến có thể được khai báo bên trong

```
{
    số nguyêntemp =
    x+y ; z = foo ( temp ) ;
}
```

- Khối có thể trống {}
- Không có dấu chấm phẩy ở cuối

Khối lồng nhau

- Các khối lồng vào nhau

```
{
    số nguyênnhiệt độ = x + y;
    z   = foo ( temp ) ;
    {
        trôi nổinhiệt độ2 = x*y ;
        z += bar ( temp2 ) ;
    }
}
```

- Ôn tập
- Khối và câu phức
- Luồng điều khiển
  - Câu lệnh điều kiện
  - Vòng lặp
- Chức năng
- Lập trình mô-đun
- Phạm vi biến đổi
  - Biến tĩnh
  - Đăng ký biến

Điều kiện kiểm soát

- Không giống như C++ hay Java, không có *boolean* loại (trong C89/C90)
  - Trong C99, kiểu dữ liệu boolean có sẵn (sử dụng stdbool.h)
- Điều kiện là một biểu thức (hoặc một chuỗi biểu thức)
  ví dụ n < 3 hoặc x < y || z < y
- Biểu thức này khác không ⇒ điều kiện đúng
- Biểu thức phải là số (hoặc con trỏ)

```
hằng char      str [ ] = "một số văn bản";
nếu như (str )   /*  chuỗi không rỗng */
    trở lại    0 ;
```

Câu lệnh điều kiện

- Cáinếu nhưtuyên bố
- Cáicông tắctuyên bố

Cáinếu như**tuyên bố**

```
nếu như ( x % 2)
    y  + = x / 2 ;
```

- Đánh giá tình trạng

  nếu như(x % 2 == 0)

- Nếu đúng, hãy đánh giá câu phát biểu bên trong.

  y ± x/2;

- Nếu không, đừng làm gì cả.

Cáikhác từ khóa

```
nếu như ( x % 2 == 0)
    y  + =  x / 2 ;
khác
    y  + =  ( x + 1 ) / 2 ;
```

- Không bắt buộc
- Thực thi câu lệnh nếu điều kiện là sai

  y ± (x+1)/2;
- Một trong hai câu lệnh bên trong có thể bị chặn.

Cáingược lại nếutừ khóa

```
nếu như( x % 2 == 0)
    y  + =  x / 2 ;
khác  nếu như( x % 4 == 1)
    y  + =  2 *( (x + 3 ) / 4 ) ;
khác
    y  + =  ( x + 1 ) / 2 ;
```

- Các đường điều khiển thay thế bổ sung
- Các điều kiện được đánh giá theo thứ tự cho đến khi một điều kiện được đáp ứng; sau đó câu lệnh bên trong được thực thi.
- Nếu nhiều điều kiện cùng đúng, chỉ điều kiện đầu tiên được thực thi.
- Tương đương với lồng nhaunếu nhưtuyên bố

Làm tổnếu nhưtuyên bố

```
nếu như(x % 4 == 0)
    nếu như( x % 2 == 0)
        y  = 2 ;
khác
    y  =  1;
```

Đến mức nàonếu nhưtuyên bố đókhácTừ khóa thuộc về?

Làm tổnếu nhưtuyên bố

Để liên kếtkhácvới bên ngoàinếu nhưcâu lệnh: sử dụng dấu ngoặc nhọn

```
nếu như(x % 4 == 0) {
    nếu như( x % 2 == 0)
        y  = 2 ;
}khác
    y =    1;
```

Cáicông tắctuyên bố

- Câu điều kiện thay thế
- Biến kiểu số nguyên (hoặc ký tự) được nhập vào.
- Xem xét các trường hợp về giá trị của biến.

```
công tắc   ( ch )   {
    trường hợp  'Y' :     /*ch == 'Y'          */
        /*làm điều gì đó*/ phá
        vỡ;
    trường hợp'N' :/*ch == 'N'        */
        /*làm điều gì đó khác*/ phá vỡ
        ;
    mặc định:   /*nếu không thì*/
        /*làm điều thứ ba*/ phá vỡ;

}
```

Nhiều trường hợp

- So sánh biến số với từng trường hợp theo thứ tự.
- Khi tìm thấy kết quả phù hợp, mã bên trong sẽ bắt đầu thực thi cho đến khi phá vỡ;đạt
- Việc thực thi “thất bại” nếuphá vỡ;không bao gồm

```
công tắc ( ch ) {
    trường hợp 'Y' :
    trường hợp 'y' :
        /* LÀMthứ gì đó nếu như
           ch == 'Y' hoặc
           ch == ' y ' */
        phá vỡ;
}
```

```
công tắc ( ch ) {
    trường hợp 'Y' :
        /* LÀMthứ gì đó nếu như
           ch == 'Y'*/
    trường hợp 'N' :
        /* LÀMthứ gì đó nếu như
           ch == 'Y' hoặc
           ch == 'N ' */
        phá vỡ;
}
```

Cáicông tắctuyên bố

- Nội dung củacông tắctuyên bố một khối
- Nhãn trường hợp: các điểm vào khác nhau vào khối
- Tương tự như các nhãn được sử dụng vớiđi đếnTừ khóa (bài giảng tiếp theo...)

Câu lệnh vòng lặp

- Cáitrong khivòng lặp
- Cáivìvòng lặp
- Cáilàm trong khivòng lặp
- Cáiphá vỡVàTiếp tụctừ khóa

Cáitrong khivòng lặp

trong khi (/* tình trạng */)
/* vòng lặp thân hình */

- Cấu trúc vòng lặp đơn giản nhất – đánh giá phần thân vòng lặp miễn là điều kiện đúng.
- Tình trạng bệnh lý được đánh giá trước, vì vậy thi thể có thể không bao giờ được thi hành.

Cáivìvòng lặp

```
số nguyêngiai thừa (số nguyênN ) {
    số nguyêni, j = 1;
    vì(i = 1 ; i <= n ; i ++)
        j *=      Tôi ;
    trở lại     j ;
}
```

- Vòng lặp “đếm”
- Bên trong dấu ngoặc đơn là ba cụm từ, được phân tách bởi dấu chấm phẩy:
    - Khởi tạo:i = 1
    - Tình trạng:i <= n
    - Tăng dần:i++
- Biểu thức có thể để trống (điều kiện được giả định là “đúng”)

Cáivìvòng lặp

Tương đương vớitrong khivòng lặp:

```
số nguyêngiai thừa (số nguyênN ) {
    số nguyênj = 1 ;
    số nguyêni = 1 ;/*khởi tạo*/ trong khi(i <=
    n/*tình trạng*/) {
        j *=      Tôi ;
        i ++;     /*   sự gia tăng*/
    }
    trở lại     j ;
}
```

Cáivìvòng lặp

- Các cụm từ ghép được phân tách bằng dấu phẩy

```
số nguyêngiai thừa (số nguyênN ) {
    số nguyêni, j;
    vì(i = 1 , j = 1 ; i <= n ; j*= i , i ++)
        ;
    trở lạij ;
}
```

- Dấu phẩy: toán tử có độ ưu tiên thấp nhất, được đánh giá từ trái sang phải; không giống như dấu phẩy giữa các đối số hàm.

Cáilàm trong khivòng lặp

```
ký tự   c ;
LÀM{
    /*   thân vòng lặp*/
    đặt s ("Tiếp tục chứ? (có/không) ; c =
    get char ( ) ;
    /*quá trình xử lý khác*/
}trong khi(c == 'y' &&/*các điều kiện khác*/);
```

- Khác vớitrong khivòng lặp – điều kiện được đánh giá sau mỗi lần lặp
- Thi thể bị hành quyết ít nhất một lần
- Lưu ý dấu chấm phẩy ở cuối

Cáiphá vỡtừ khóa

- Đôi khi muốn kết thúc vòng lặp sớm.
- **phá vỡ**;thoát khỏi vòng lặp trong cùng hoặccông tắctuyên bố rời đi sớm

- Hãy xem xét việc sửa đổi củalàm trong khiví dụ:

```
ký tự   c ;
LÀM{
    /*   thân vòng lặp*/
    đặt s ("Tiếp tục chứ? (có/không) ; c =
    get char ( ) ; nếu như(c ! =
                    'y')
        phá vỡ;
    /*khác        xử lý*/
}   trong khi(/*    các điều kiện khác*/);
```

Cái Tiếp tục từ khóa

- Sử dụng để bỏ qua một lần lặp
- **Tiếp tục**;bỏ qua phần còn lại của thân vòng lặp trong cùng, nhảy đến điều kiện vòng lặp.
- Ví dụ:

```
# định nghĩa min ( a , b ) ( ( a ) < ( b ) ? ( a ) : ( b ) )

số nguyên ước chung lớn nhất (số nguyên , số nguyên b) {
    số nguyên tôi, trả lại = 1, minva l = min ( a , b ) ;
    vì (i = 2 ; i <= minval ; i ++) {
        nếu như (a % i )/*tôi không phải là ước số của*/
            Tiếp tục;
        nếu như (b % i == 0)/*i là ước số của cả a và b*/
            trả lại = i ;
    }
    trở lại trả lại;
}
```

- Ôn tập
- Khối và câu phức
- Luồng điều khiển
  - Câu lệnh điều kiện
  - Vòng lặp
- Chức năng
- Lập trình mô-đun
- Phạm vi biến đổi
  - Biến tĩnh
  - Đăng ký biến

Chức năng

- Tôi đã thấy một số chức năng, bao gồm:chủ yếu():

```
số nguyên chủ yếu (khoảng trống)  {
    /*  LÀMchất liệu    */
    trở lại    0 ;/*    thành công*/
}
```

- Cú pháp cơ bản của các hàm được giải thích trong Bài giảng 1.
- Làm thế nào để viết chương trình sử dụng hàm?

Chia để trị

- Hãy hình dung cách một chương trình có thể được chia thành các phần nhỏ hơn.
- Hãy thiết kế một chương trình để giải phương trình Diophantine tuyến tính (*rìu+qua=c,x, y*: số nguyên):

  Lấy a, b, c từ dòng lệnh tính g = gcd(a,b)

  nếu như (*c không phải là bội số của ước chung lớn nhất*)

  Không có giải pháp nào; thoát

  Chạy thuật toán Euclid mở rộng trên a, b, điều chỉnh tỷ lệ x và y đầu ra bằng (c/g)

  giải pháp in ấn

- Thuật toán Euclid mở rộng: tìm số nguyên *x, y* st

rìu+qua=ước chung lớn nhất (*a, b*).

Tính ước chung lớn nhất

- Tính ước chung lớn nhất (GCD) bằng thuật toán Euclid:

```
số nguyên ước chung lớn nhất (số nguyên Một ,  số nguyên b)  {
    trong khi (b)  {  /*  nếu như Một < b , mỗi lần hoán đổi hình thức */
        số nguyên nhiệt độ
        = b; b = a % b;
        a =  nhiệt độ ;
    }
    trở lại  Một ;
}
```

- Thuật toán dựa trên ước chung lớn nhất (a, b) = gcd(b, a mod b), đối với số tự nhiên $a > b$.

[Knuth, DE Nghệ thuật lập trình máy tính, Tập 1: Cơ bản]

Thuật toán. Ấn bản lần thứ 3. Addison-Wesley, 1997.]

© Addison Wesley. Mọi quyền được bảo lưu. Nội dung này không thuộc phạm vi giấy phép Creative Commons của chúng tôi. Để biết thêm thông tin, vui lòng xem http://ocw.mit.edu/fairuse.

Thuật toán Euclid mở rộng

Mã giả cho thuật toán Euclid mở rộng:

Khởi tạo các biến trạng thái (x, y) nếu như (*a < b*)

hoán đổi (a, b)

trong khi (*b > 0*) {

Tính thương, cập nhật số dư, cập nhật các biến trạng thái (x, y)

}

Trả về ước chung lớn nhất (gcd) và các biến trạng thái (x, y)

[Menezes, AJ, et al. Cẩm nang mật mã ứng dụng. Nhà xuất bản CRC Press, 1996.]

© CRC Press. Bảo lưu mọi quyền. Nội dung này không thuộc phạm vi giấy phép Creative Commons của chúng tôi. Để biết thêm thông tin, vui lòng xemhttp://ocw.mit.edu/fairuse.

Trả về nhiều giá trị

- Thuật toán Euclid mở rộng trả về ước chung lớn nhất (gcd) và hai biến trạng thái khác, x và y.
- Hàm chỉ trả về (tối đa) một giá trị.
- Giải pháp: sử dụng *toàn cầu* biến số
- Khai báo các biến cho các đầu ra khác bên ngoài hàm.
  - Các biến được khai báo bên ngoài khối hàm được gọi là biến toàn cục.
  - tồn tại trong suốt vòng đời của chương trình.
  - có thể được truy cập/sửa đổi trong bất kỳ hàm nào.

Chia để trị

- Chia nhỏ vấn đề thành các vấn đề nhỏ hơn.
- Hãy xem xét phép lặp và đệ quy.
  - Làm thế nào chúng ta có thể triển khai gcd(a,b) một cách đệ quy?
- Giảm thiểu việc truyền trạng thái giữa các hàm.
- Việc viết mã giả trước có thể giúp ích.

- Ôn tập
- Khối và câu phức
- Luồng điều khiển
  - Câu lệnh điều kiện
  - Vòng lặp
- Chức năng
- Lập trình mô-đun
- Phạm vi biến đổi
  - Biến tĩnh
  - Đăng ký biến

Lập trình các mô-đun bằng ngôn ngữ C.

- Các chương trình C không nhất thiết phải có cấu trúc nguyên khối.
- Mô-đun: giao diện và triển khai
  - giao diện: các tệp tiêu đề
  - triển khai: các tệp nguồn/đối tượng phụ trợ
- Khái niệm tương tự cũng áp dụng cho các thư viện bên ngoài (tuần tới...).

Mô-đun Euclid

- Các thuật toán của Euclid hữu ích trong nhiều lĩnh vực.
- Tôi muốn tích hợp chức năng này vào nhiều chương trình.
- Giải pháp: tạo một mô-đun cho các thuật toán Euclid
- Cần viết tệp tiêu đề (.h)và tệp nguồn (.c)

Nguồn:euclid.c

Thực hiệngcd()TRONGeuclid.c:

```
/*  Ước chung lớn nhất ( )  chức năng  */
số nguyên  ước chung lớn nhất (số nguyên  Một ,  số nguyên  b)  {
    trong khi  ( b )  {  /*  nếu như  Một > b , theo các biểu mẫu  tráo đổi  */
        số nguyên nhiệt độ
        = b; b = a % b;
        a =  nhiệt độ ;
    }
    trở lại  Một ;
}
```

Thuật toán Euclid mở rộng được triển khai như sau: ext_euclid(),cũng ởeuclid.c

Cáibên ngoàitừ khóa

- Cần thông báo cho các tập tin nguồn khác về các hàm/biến toàn cục trongeuclid.c
- Đối với các hàm: hãy đặt nguyên mẫu hàm vào một tệp tiêu đề.
- Đối với các biến: hãy khai báo lại biến toàn cục bằng cách sử dụng bên ngoàitừ khóa trong tệp tiêu đề
- bên ngoàiThông báo cho trình biên dịch rằng biến đó được định nghĩa ở nơi khác.
- Cho phép truy cập/sửa đổi biến toàn cục từ các tệp nguồn khác.

Tiêu đề:euclid.h

Phần tiêu đề chứa các nguyên mẫu chogcd()Vàext_euclid():

```
/*đảm bảo     bao gồm chỉ một lần*/
# nếu nde f        __EUCLID_H__
# định nghĩa    __EUCLID_H__

/*toàn cầu     biến ( dec l ared ineuclid . c )*/
bên ngoài       x, y;

/*tính ước chung lớn nhất*/
số nguyênước chung lớn nhất (số nguyênMột ,số nguyênb) ;

/*tính g = gcd ( a , b ) và do đó ta có ax+by=g*/ số nguyênext _
euclid (số nguyênMột ,số nguyênb) ;

# endi f
```

Sử dụng mô-đun Euclid

- Muốn có thể gọi điệngcd()hoặcext_euclid()từ tệp chính diophant.c
- Cần phải bao gồm tệp tiêu đềeuclid.h:

  # bao gồm"euclid.h"(fi(le trong “.”, không phải đường dẫn tìm kiếm)

- Sau đó, có thể gọi như bất kỳ hàm nào khác:

```
/*tính g = gcd ( a , b )*/ g = gcd ( a ,
b ) ;

/*Tính x và y bằng cách sử dụng đại số Euclid mở rộng.*/ g = ext _
euclid ( a , b ) ;
```

- Kết quả trong các biến toàn cầuxVày

```
/*Điều chỉnh tỷ lệ sao cho ax + by = c*/ phát
triển    = c / g ;
x   *=   phát triển ;
y   *=   phát triển ;
```

Biên dịch với mô-đun Euclid

- Chỉ đang biên soạndiophant.ckhông đủ
- Các chức nănggcd()Vàext_euclid()được định nghĩa trong euclid.c;Tệp nguồn này cũng cần được biên dịch.
- Khi biên dịch các tập tin nguồn, các đầu ra cần được liên kết với nhau thành một đầu ra duy nhất.
- Một cuộc gọi tớigcccó thể hoàn thành tất cả những điều này:

```
Athena%[1]gcc -g -O0 -Wall diophant.c euclid.c -o
diophant.o
```

- diophant.ocó thể hoạt động như bình thường

[1]Athena là môi trường điện toán dựa trên UNIX của MIT. OCW không cung cấp quyền truy cập vào môi trường này.

- Ôn tập
- Khối và câu phức
- Luồng điều khiển
  - Câu lệnh điều kiện
  - Vòng lặp
- Chức năng
- Lập trình mô-đun
- Phạm vi biến đổi
  - Biến tĩnh
  - Đăng ký biến

Phạm vi biến đổi

- *phạm vi*–vùng mà trong đó một biến số có giá trị
- Trong nhiều trường hợp, điều này tương ứng với khối chứa khai báo biến.
- Các biến được khai báo bên ngoài hàm có phạm vi toàn cục.
- Định nghĩa hàm cũng có phạm vi.

Một ví dụ

Phạm vi ý nghĩa của mỗi biến trong ví dụ này là gì?

```
số nguyênnmax = 20;

/*Hàm chính ( )*/
số nguyênchủ yếu (số nguyênargc ,ký tự**argv )/*điểm vào*/ {

    số nguyêna = 0 , b = 1 , c , n ;
    printf ("%3d: %d\n" ,1 , a ) ;
    printf ("%3d: %d\n" ,2 , b ) ;
    vì(n = 3 ; n <= nmax ; n++) {
        c = a + b ; a = b ; b = c ; printf ("%
        3d: %d\n" ,n, c);
    }
    trở lại0 ;/*thành công*/
}
```

Phạm vi và khai báo lồng nhau

Hiện tại đã in được bao nhiêu dòng?

```
số nguyênnmax = 20;

/*Hàm chính ( )*/
số nguyênchủ yếu (số nguyênargc ,ký tự**argv )/*điểm vào*/ {

    số nguyêna = 0 , b = 1 , c , n , nmax = 25;
    printf ("%3d: %d\n" ,1 , a ) ; printf ("%3d:
    %d\n" ,2 , b ) ;
    vì(n = 3 ; n <= nmax ; n++) {
        c = a + b ; a = b ; b = c ; printf ("%
        3d: %d\n" ,n, c);
    }
    trở lại0 ;/*thành công*/
}
```

Biến tĩnh

- tĩnhTừ khóa này có hai nghĩa, tùy thuộc vào nơi biến tĩnh được khai báo.
- Bên ngoài một hàm,tĩnhCác biến/hàm chỉ hiển thị trong phạm vi tệp đó, không hiển thị toàn cục (không thể là...)bên ngoài(ed)
- Bên trong một hàm,tĩnhbiến số:
  - vẫn thuộc phạm vi cục bộ của hàm đó.
  - Chúng chỉ được khởi tạo trong quá trình khởi tạo chương trình.
  - không được khởi tạo lại với mỗi lần gọi hàm.

```
staticintsomePers i stentVar = 0 ;
```

Biến đăng ký

- Trong quá trình thực thi, dữ liệu được xử lý trong*sổ đăng ký*
- Lưu trữ dữ liệu thường dùng một cách rõ ràng trong các thanh ghi – giảm thiểu chi phí tải/lưu trữ.
- Có thể khai báo rõ ràng một số biến nhất định là thanh ghi bằng cách sử dụng đăng kýtừ khóa
  - phải là một kiểu dữ liệu đơn giản (tùy thuộc vào cách triển khai)
  - Chỉ các biến cục bộ và đối số hàm mới đủ điều kiện.
  - Các khai báo thanh ghi thừa/không được phép sẽ bị bỏ qua, được biên dịch như các biến thông thường.
- Các thanh ghi không nằm trong vùng nhớ được địa chỉ hóa; con trỏ của một biến thanh ghi là bất hợp pháp.

Ví dụ

Ví dụ về phạm vi biến, xem xét lại, vớiđăng kýbiến số:

```
/*Hàm chính ( )*/
số nguyênchủ yếu (đăng kýargc ,đăng ký ký tự**argv ) {

    đăng kýa = 0 , b = 1 , c , n , nmax = 20; printf ("%3d:
    %d\n" ,1 , a ) ; printf ("%3d: %d\n" ,2 , b ) ;

    vì(n = 3 ; n <= nmax ; n++) {
        c = a + b ; a = b ; b = c ; printf ("%
        3d: %d\n" ,n, c);
    }
    trở lại0 ;/*thành công*/
}
```

Bản tóm tắt

Các chủ đề được đề cập:

- Kiểm soát luồng chương trình bằng cách sử dụng câu lệnh điều kiện và vòng lặp.
- Chia một chương trình phức tạp thành nhiều chương trình con đơn giản hơn bằng cách sử dụng các hàm và kỹ thuật lập trình module.
- Quy tắc phạm vi biến vàbên ngoài, tĩnh,Và đăng kýbiến số

MIT OpenCourseWare
http://ocw.mit.edu

6.087 Lập trình thực hành bằng ngôn ngữ C
Tháng 1 (IAP) 2010

Để biết thông tin về cách trích dẫn các tài liệu này hoặc Điều khoản sử dụng của chúng tôi, vui lòng truy cập:http://ocw.mit.edu/terms.